

AI in signal and audio processing

Thomas de Jaeger

thomas.de-jaeger@epita.fr

Syllabus

- **Session 1:** Introduction & Fundamentals of Signal/Audio
- **Session 2:** Audio Features & Librosa
- **Session 3:** Audio Processing with Torchaudio
- **Session 4:** Sound Detection and Classification
- **Session 5:** Sequence Modeling with RNNs and LSTMs
- **Session 6:** Automatic Speech Recognition (ASR)
- **Session 7:** Speaker and Emotion Recognition
- **Session 8:** Project presentation

Audio Features & Librosa

Session 2

Why Preprocess Audio?

Goal: Prepare raw audio data to be more meaningful and robust for AI models.

Raw audio is often:

- Noisy
- Different durations, loudness, background
- Variable in volume or silence
- Waveforms don't reveal pitch, energy, or phonemes directly

ML models need consistent, compact, and informative representation

Audio Preprocessing Pipeline

A typical audio preprocessing pipeline includes:

- **Resampling** – Make all audio signals use the same sampling rate.
- **Mono Conversion** – Standardize to one channel.
- **Silence Trimming** – Remove uninformative quiet sections.
- **Normalization** – Ensure consistent volume.
- **Framing & Windowing** – Divide into frames for temporal analysis.
- **Feature Extraction** – Extract ZCR, MFCCs (this session).

Resampling

Why Resample Audio?

- Audio files may have different sampling
- ML models require consistent input shape — we resample all audio to a standard rate (e.g., 16,000 or 22,050 Hz).

What is Sampling Rate?

- Number of audio samples per second.
- Nyquist-Shannon theorem: sampling rate must be $\geq 2\times$ the highest frequency in the signal.

In Python (librosa):

```
y, sr = librosa.load("audio.wav", sr=16000)
```

#librosa.load() handles resampling automatically when sr is specified.

Note:

Choose `sr = 16000` for voice-only tasks.

Use `sr = 22050` or `sr = 44100` for music/audio classification

Mono Conversion

Why convert to mono?

- Stereo files have 2 channels (left/right), which doubles the input shape.
- Most speech/audio tasks don't require spatial information → convert to mono (1 channel).

What Happens?

- Combines left and right channels:

$$y_{\text{mono}} = \frac{1}{2}(y_L + y_R)$$

In Python (librosa):

- `y, sr = librosa.load("audio.wav", mono=True)`

When to keep stereo?

- Music spatialization, localization, binaural audio, etc.

Silence Trimming

Why Remove Silence?

- Long leading/trailing silences are non-informative.
- They skew statistics and waste computation.

Particularly important for:

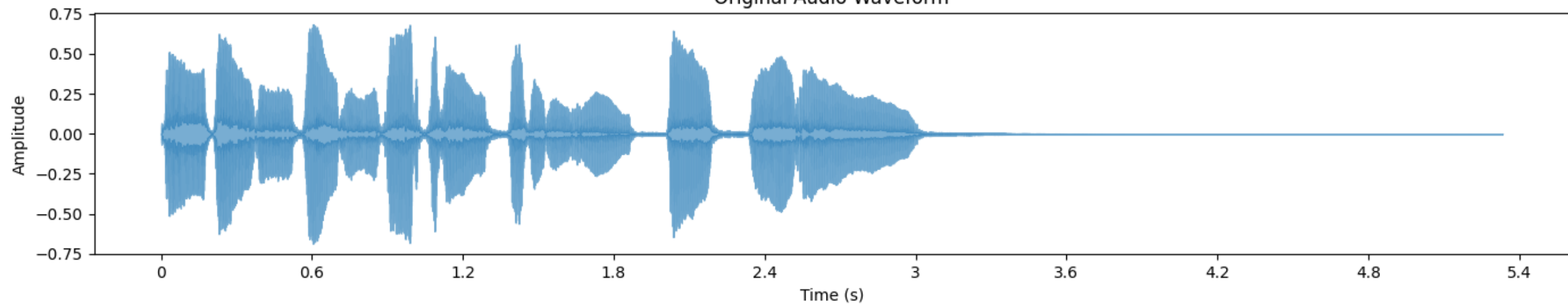
- Speaker ID
- Emotion recognition
- Classification

In Python (librosa):

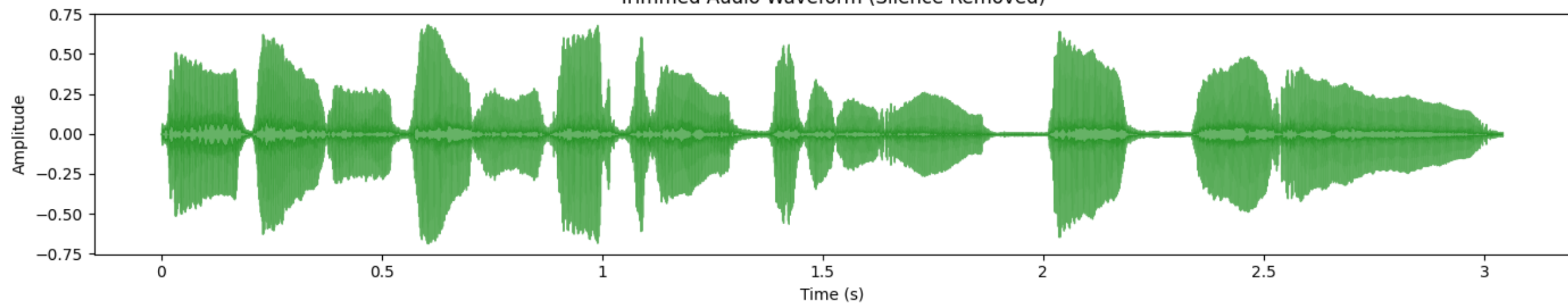
`y_trimmed, index = librosa.effects.trim(y, top_db=20)` # uses an energy-based threshold to detect silence.
top_db: how aggressive the silence removal is (higher = stricter).

Silence Trimming

Original Audio Waveform



Trimmed Audio Waveform (Silence Removed)



Normalization

Rescales signal so max amplitude = 1.0

- Audio files can have different loudness (amplitude).
- ML models may become biased toward louder inputs.
- usually common method: peak normalization $y' = y / \max(|y|)$

Task	Normalize?	Why
Speech Recognition	✓	Avoid bias from microphone gain
Music Genre Classification	✓	Ensures dynamics don't dominate
Voice Emotion Recognition	⚠ maybe	Some features use loudness info
Speaker Identification	⚠ maybe	Loudness may be speaker-specific

Framing

Framing is the process of dividing an audio signal into short overlapping segments called *frames*, so we can analyze how the signal changes over time.

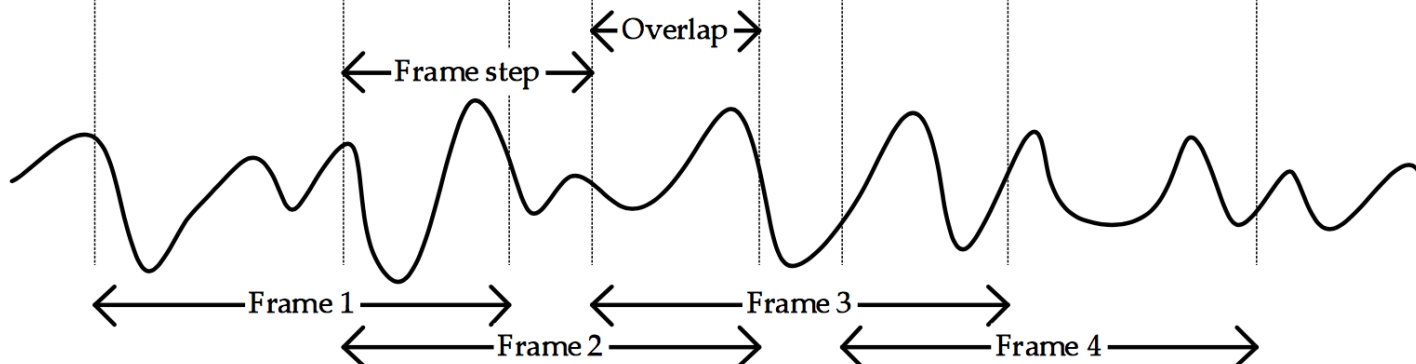
A frame is a short, fixed-length segment of an audio signal, typically lasting between 10 to 50 milliseconds.

- Audio is non-stationary, meaning its properties change over time.
- But within a short frame, we can assume the signal is quasi-stationary → we can extract meaningful features (RMS, MFCC, etc.)

Parameter	Value
Frame Size	256 – 2048 samples (power of 2)
Frame Duration	25 ms (speech), 50 ms (music)
Sampling Rate	16,000 Hz
Hop Length	50% overlap (e.g. 512 hop for 1024 frame)

$$\text{Frame duration (s)} = \frac{\text{Frame size}}{\text{Sampling rate}}$$

=how much we move forward to get the next frame
Smaller hop = more overlap = better temporal resolution



Windowing

Each frame is usually multiplied by a **window function** before analysis to reduce edge effects (spectral leakage).

Common windows:

$$x_w[n] = x[n] \cdot w[n]$$

- Hamming: use for **time-domain energy tracking**, **speech onset detection**, or when using **short frames** (e.g., voice activity)

$$w[n] = 0.5 \left(1 - \cos \left(\frac{2\pi n}{N-1} \right) \right)$$

- Hann: use when you need **good frequency resolution** and **reduced spectral leakage**, such as for computing spectrograms or MFCCs. y detection)

$$w[n] = 0.54 - 0.46 \cos \left(\frac{2\pi n}{N-1} \right)$$

- Blackman: use for **music signal analysis**, **pitch detection**, and **applications where frequency content must be isolated cleanly**, but at the cost of frequency resolution.

$$: w[n] = 0.42 - 0.5 \cos \left(\frac{2\pi n}{N-1} \right) + 0.08 \cos \left(\frac{4\pi n}{N-1} \right)$$

Windowing

```
# Create window functions
```

```
hann = np.hanning(frame_length)
```

```
hamming = np.hamming(frame_length)
```

```
blackman = np.blackman(frame_length)
```

```
# Apply windowing
```

```
windowed_hann = frame * hann
```

```
windowed_hamming = frame * hamming
```

```
windowed_blackman = frame * blackman
```

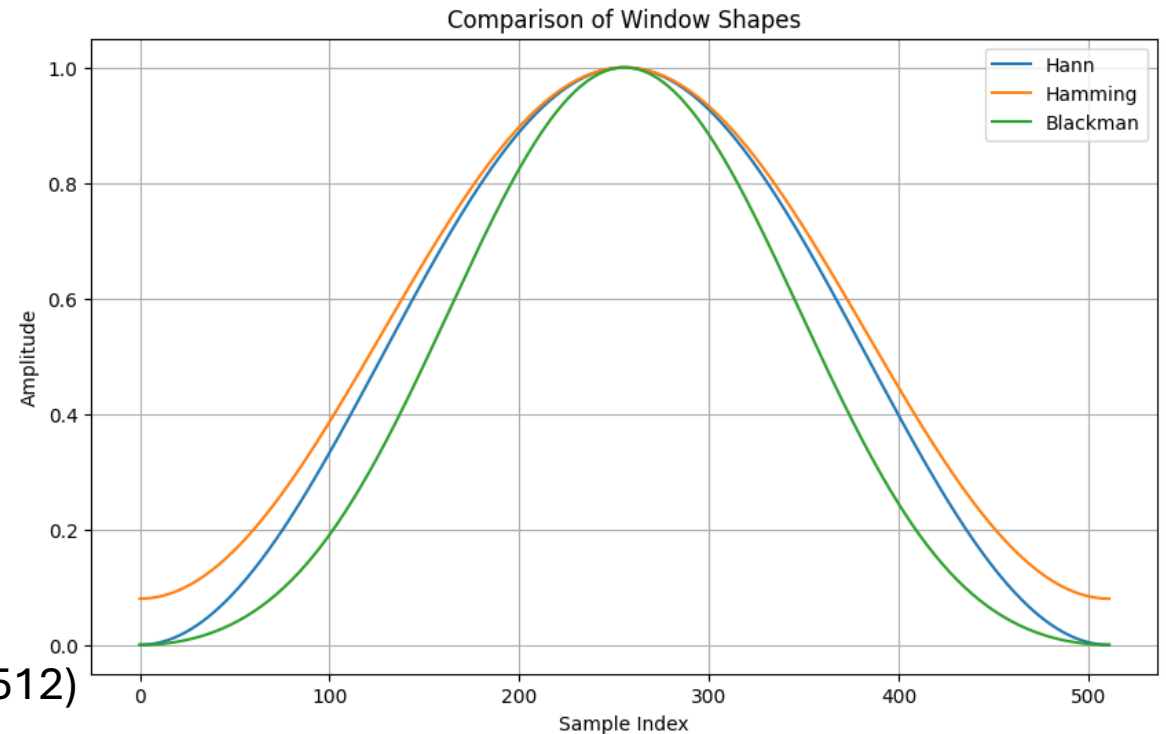
OR

```
window = librosa.filters.get_window("hann", frame_length)
```

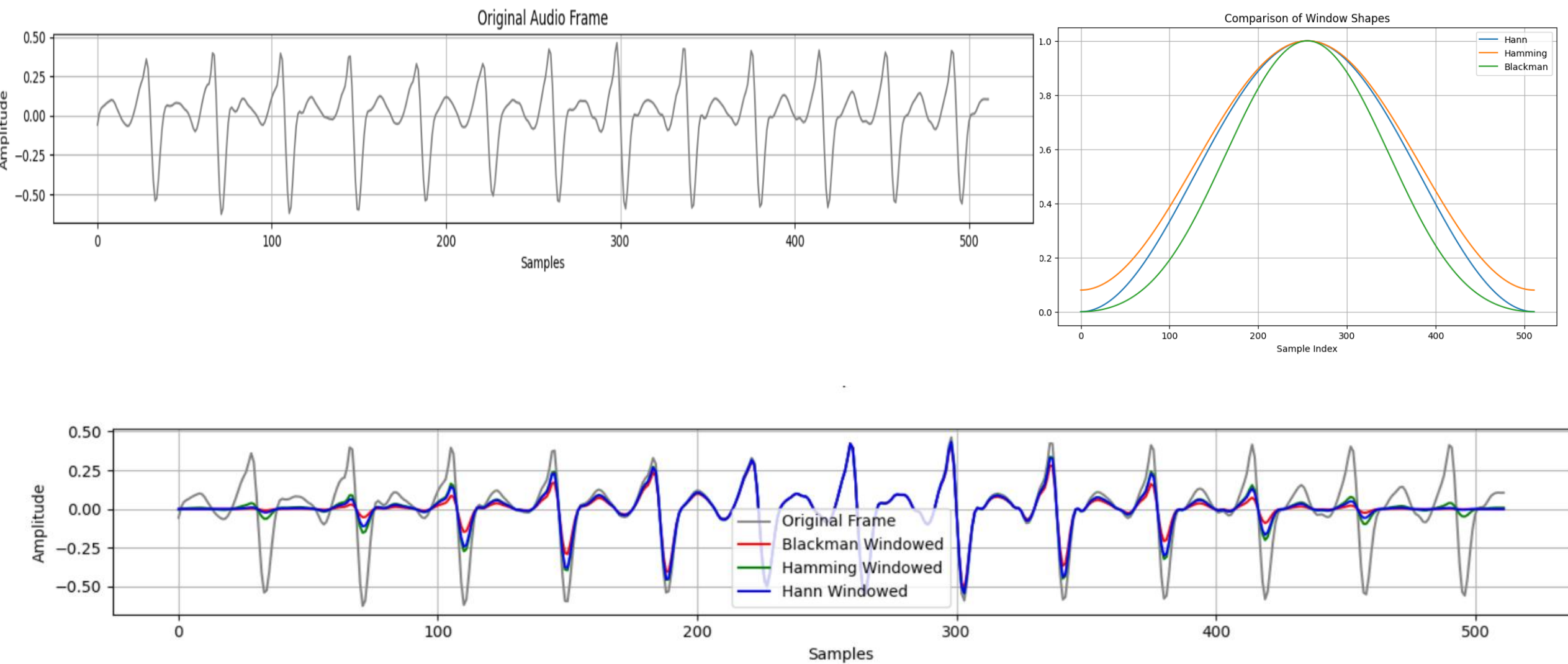
```
frames = librosa.util.frame(y, frame_length=1024, hop_length=512)
```

```
# Apply windowing to each frame
```

```
windowed_frames = frames * window[:, np.newaxis]
```



Windowing



Harmonic/Percussive Separation

Purpose:

Separate harmonic (tonal, sustained) and percussive (transient, sharp) components of an audio signal. Harmonic: instruments, vocals, stable pitches Percussive: drums, claps, transient sounds

Why do this?

- Enhance audio classification (e.g., bird songs vs. drum beats)
- Improve transcription or source separation
- Reduce noise or isolate rhythmic patterns

Method:

- Based on differences in spectrogram smoothness:
 - Harmonic: smooth across time (horizontal lines)
 - Percussive: smooth across frequency (vertical lines)

Python (librosa)

- `harmonic, percussive = librosa.effects.hpss(y)`

Harmonic/Percussive Separation

Purpose:

Separate harmonic (tonal, sustained) and percussive (transient, sharp) components of an audio signal. Harmonic: instruments, vocals, stable pitches Percussive: drums, claps, transient sounds

Why do this?

- Enhance audio classification (e.g., bird songs vs. drum beats)
- Improve transcription or source separation
- Reduce noise or isolate rhythmic patterns

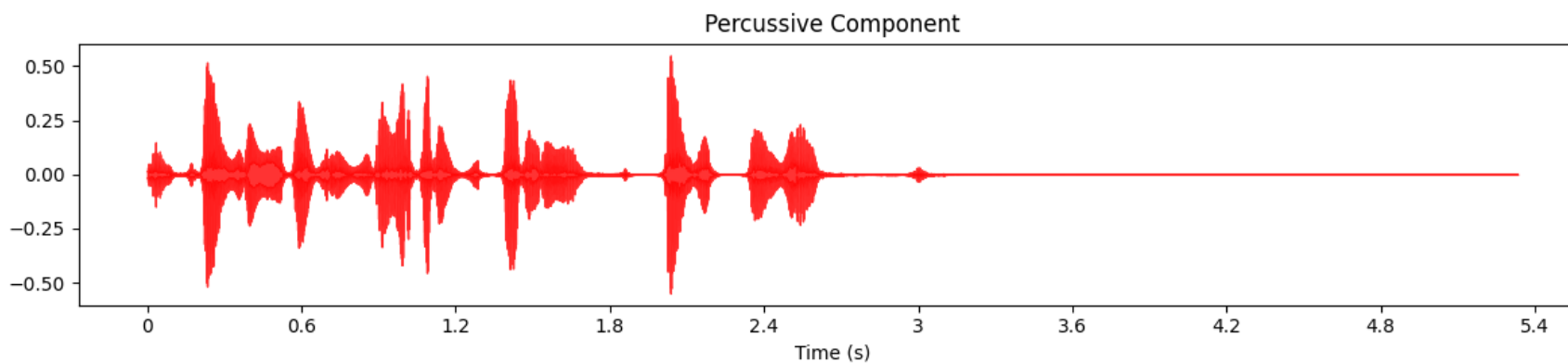
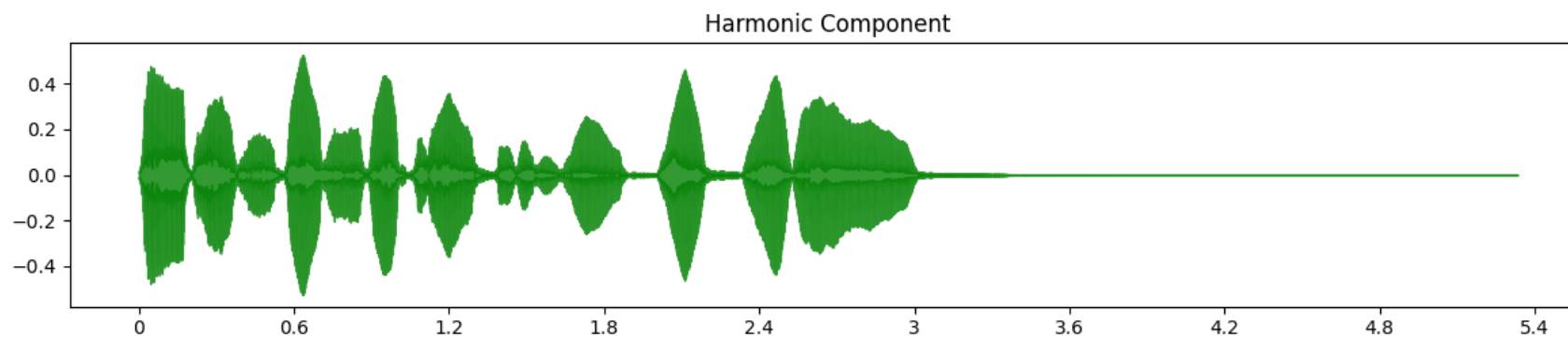
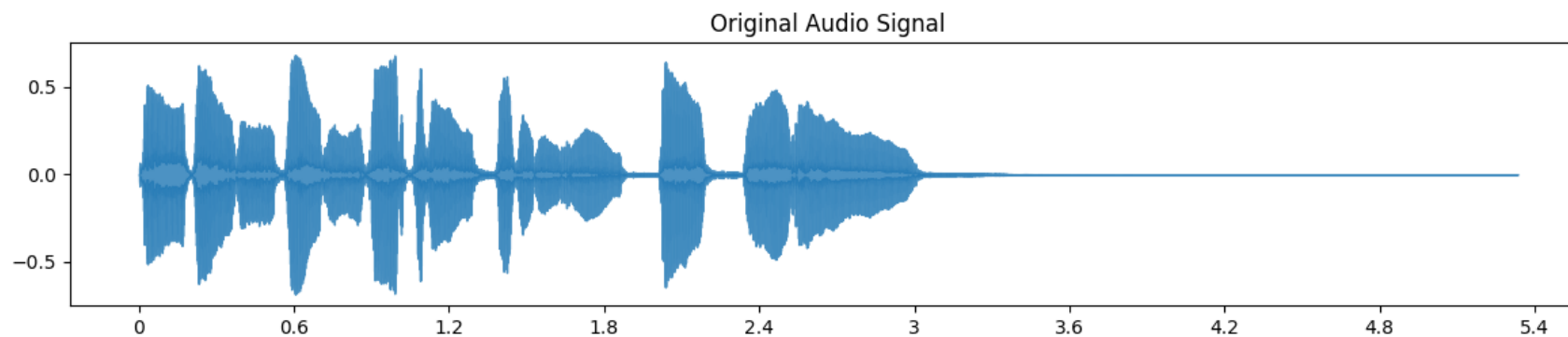
Method:

- Based on differences in spectrogram smoothness:
 - Harmonic: smooth across time (horizontal lines)
 - Percussive: smooth across frequency (vertical lines)

Python (librosa)

- `harmonic, percussive = librosa.effects.hpss(y)`

Harmonic/Percussive Separation



Feature extraction

Once audio is preprocessed, we extract descriptive features to represent the signal. Each feature captures a different signal property — energy, rhythm, pitch, timbre.

- **Time Domain**
 - Amplitude Envelope
 - Root Mean Square Energy (RMS)
 - Zero-Crossing Rate (ZCR)
 - Beat
- **Frequency Domain**
 - Band Energy Ratio
 - Spectral Centroid
 - Spectral Flux
 - Spectral Spread
 - Fourier transformation
- **Time-Frequency Domain**
 - Spectrogram
 - Mel Spectrogram
 - MFCC (Mel-Frequency Cepstral Coefficients)
 - Constant-Q Transform (CQT)

Time Domain: Amplitude Envelope

Tracks the outer shape of the waveform, giving a sense of how loud the signal is at each moment.

- Captures the dynamic range and loudness variation
- Useful for segmenting audio (e.g., separating notes, detecting speech activity)
- Cons: It is sensitive to outliers

How it works?

Divide the signal into short frames, then compute the maximum absolute amplitude for each frame.

$$AE(t) = \max_{n \in \text{frame}} |x[n]|$$

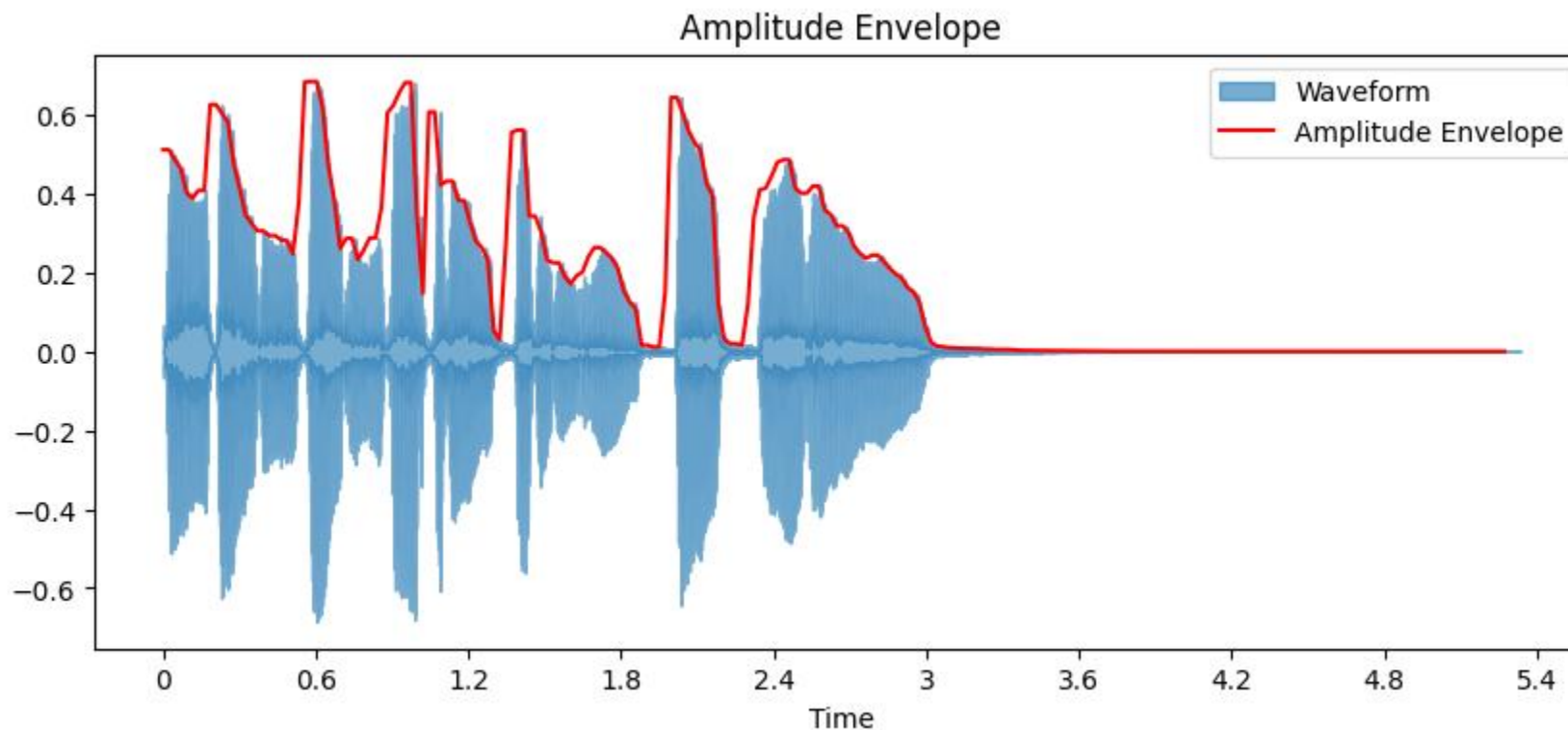
Use Cases:

- Onset detection in music
- Music classification
- Detecting quiet vs. loud segments in a recording

Time Domain: Amplitude Envelope

Python (librosa)

- `frames = librosa.util.frame(y, frame_length=frame_length, hop_length=hop_length)`
- `amplitude_envelope = np.max(np.abs(frames), axis=0)`



Time Domain: RMS

Measures the energy or loudness of the signal over time, accounting for fluctuations across each frame.

- Reflects short-term energy changes
- Less sensitive to outliers

$$\text{RMS}(t) = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} x[n]^2}$$

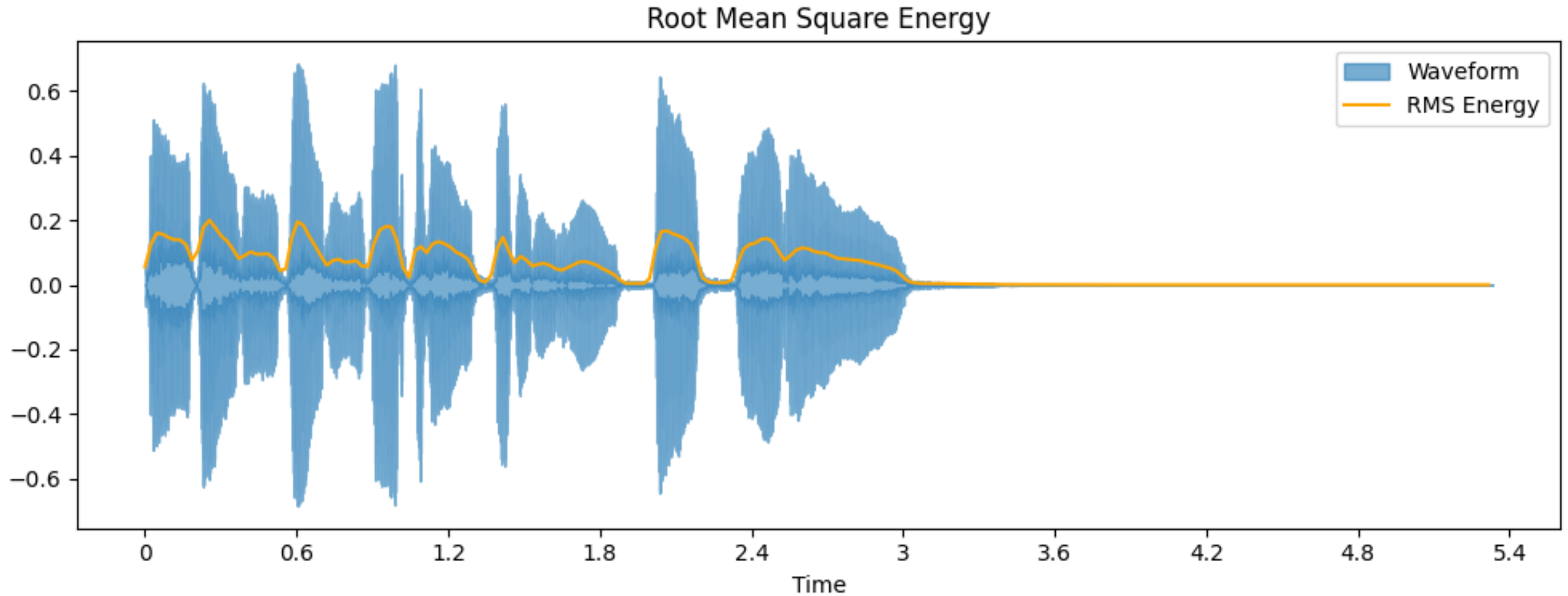
Use Cases:

- Dynamic range analysis
- Silence detection
- Energy-based segmentation (e.g., music/speech detection)

Python (Librosa)

- `signal_rms = librosa.feature.rms(y=signal, frame_length=FRAME_SIZE, hop_length=HOP_LENGTH)[0]`

Time Domain: RMS



Time Domain: Zero Crossing Rate

ZCR measures the rate at which a signal changes sign (i.e., crosses the zero axis). It gives insight into the noisiness or frequency content of a signal

- High ZCR: noisy or percussive sounds
- Low ZCR: harmonic, tonal, or smooth audio

$$ZCR = \frac{1}{T} \sum_{t=1}^T |s(t) - s(t-1)|$$

Where $s(t) = 1$ if s is positive

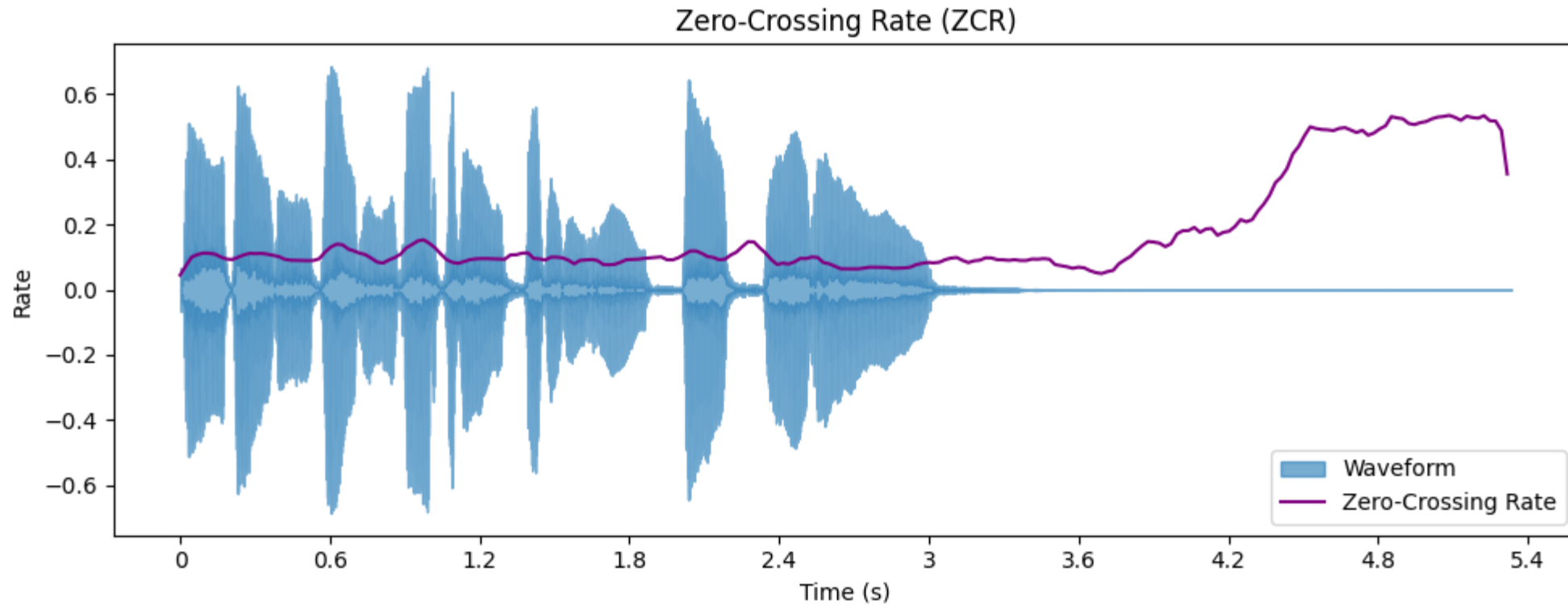
Use Cases:

- speech processing
- music recognition.

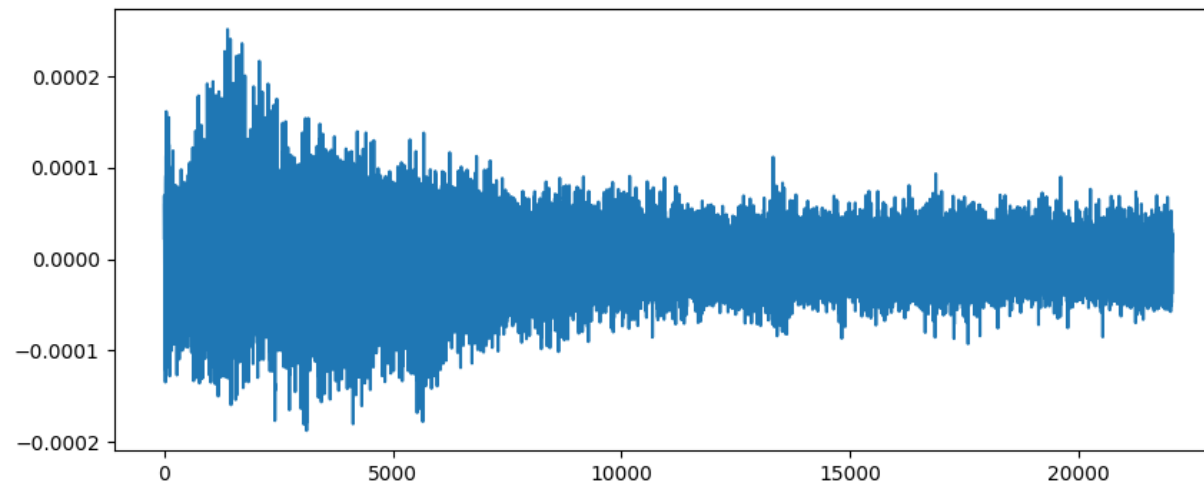
Python (Librosa)

- `zcr = librosa.feature.zero_crossing_rate(y, hop_length=hop_length)[0]`

Time Domain: Zero Crossing Rate



Time Domain: Zero Crossing Rate



Time Domain: Beat tracking

Beat tracking estimates the tempo and detects where beats occur in a piece of music.

How it works:

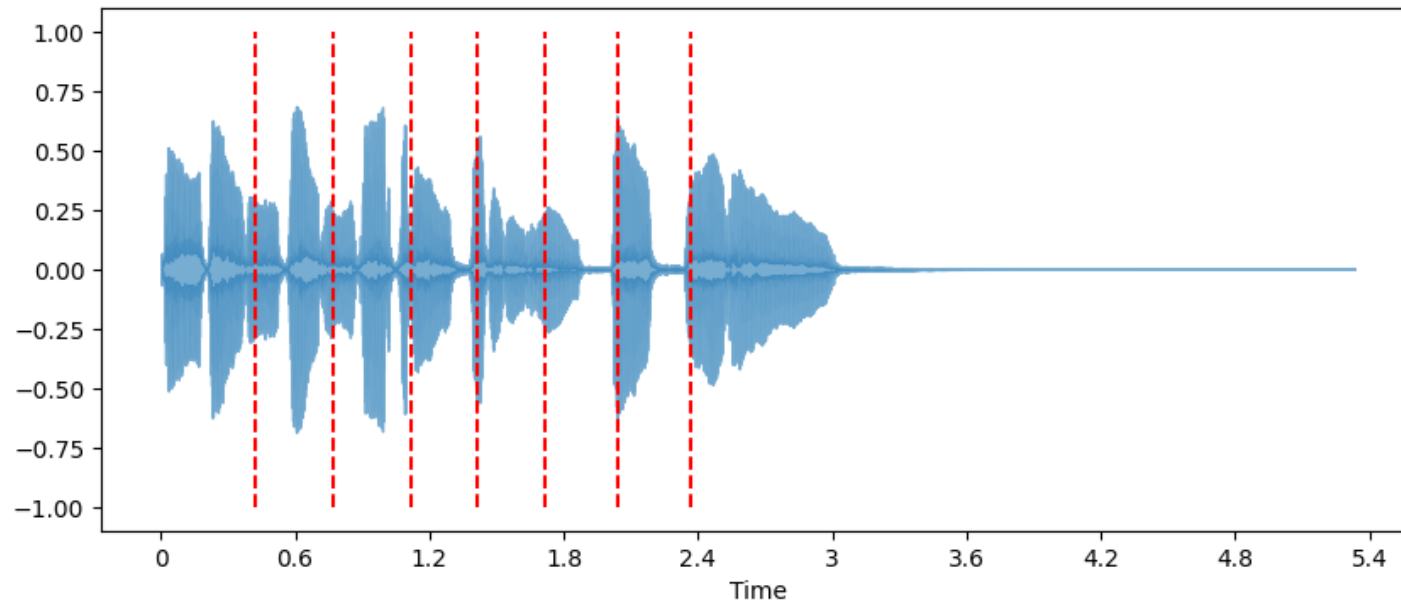
- Uses onset detection (sharp changes in energy)
- Applies dynamic programming or autocorrelation to find periodicities
- Produces a sequence of beat timestamps

Use Cases:

- Tempo estimation for DJ/music apps
- Audio segmentation (chorus, verse)
- Rhythm-aware music generation

Python (librosa)

- `tempo, beats = librosa.beat.beat_track(y=y, sr=sr)`
- `beat_times = librosa.frames_to_time(beats, sr=sr)`



Frequency Domain: FFT

Purpose: To convert a time-domain signal into its frequency-domain representation. The Fourier Transform breaks the signal into a sum of sinusoids. It tells you **how much** of each frequency exists in the signal and **how it is phased**.

Why use it?

- Understand the spectral content
- Detect dominant frequencies or harmonics
- Pre-requisite for most frequency-domain features

$$X(f) = \int_{-\infty}^{\infty} x(t) \cdot e^{-i2\pi ft} dt$$

High-Level

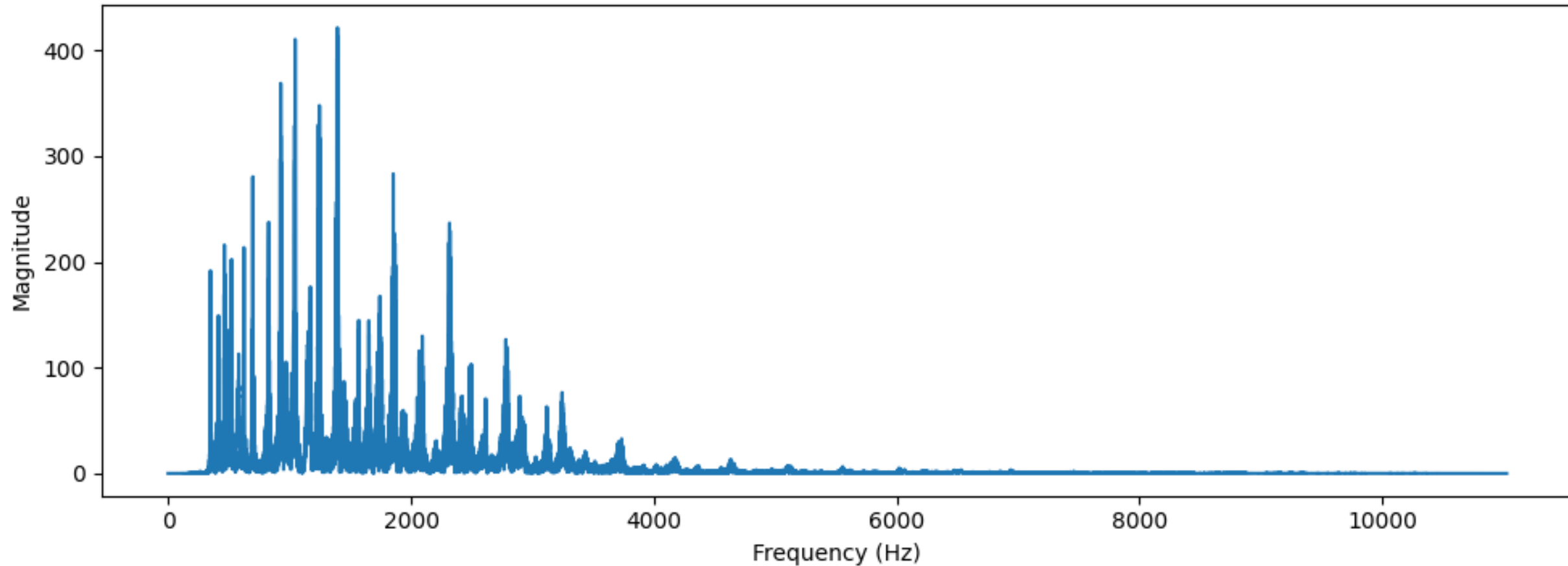
- The signal is compared to many sine waves with different frequencies.
- For each one, we measure **how strong and aligned** the match is — this gives us a **magnitude** and a **phase**.
- After choosing the best frequencies, we fine-tune the **phase** to better match the signal.
- Finally, we look at the **magnitude**:
 - A **high magnitude** means the signal is similar to that sine wave.

Python (numpy)

- `fft_result = np.fft.fft(y)`
- `magnitude = np.abs(fft_result)`
- `freq = np.fft.fftfreq(len(magnitude), 1/sr)`

Frequency Domain: FFT

Magnitude Spectrum (FFT)



Frequency Domain: Band Energy Ratio

Purpose:

Band Energy Ratio measures how the energy of a signal is distributed across low and high frequencies. It helps distinguish tonal vs. noisy or percussive content. Useful for speech music discrimination, music genre classification, mood classification

How it works:

- Apply Short-Time Fourier Transform (splits the signal into overlapping chunks, applies a window function, computes the FFT on each chunk,
- Divide the frequency spectrum into two bands: low and high
- Compute the energy ratio between the two bands for each frame

$$BER(t) = \frac{\sum_{f=0}^{f_c} |X(t, f)|^2}{\sum_{f=f_c+1}^F |X(t, f)|^2}$$

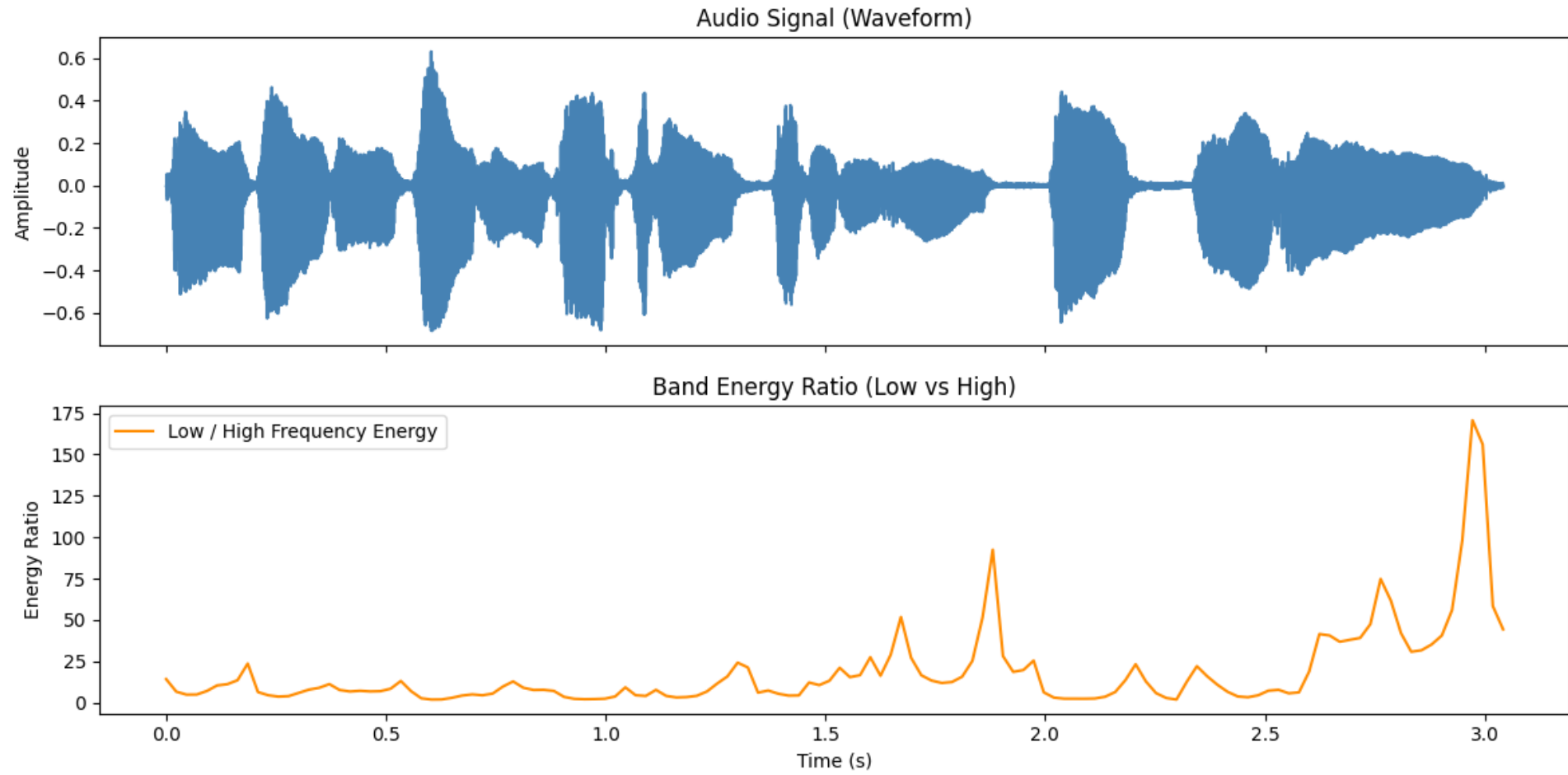
Where:

- $X(t, f)$ is the STFT of the signal
- f_c , is the cutoff frequency separating low and high bands
- F is the maximum frequency bin

Python (librosa):

```
STFTS = np.abs(librosa.stft(y))**2
freqs = librosa.fft_frequencies(sr=sr)
f_c = 2000
low_band = S[freqs <= f_c, :]
high_band = S[freqs > f_c, :]
ber = np.sum(low_band, axis=0) / (np.sum(high_band, axis=0) + 1e-6)
```

Frequency Domain: Band Energy Ratio



Frequency Domain: Spectral Centroid

Purpose:

The spectral centroid indicates where the “center of mass” of the frequency spectrum is, i.e., the frequency band where the most audio signal energy is concentrated (sound’s brightness).

Why it’s useful:

- High spectral centroid: bright or sharp sounds (e.g., cymbal, bird chirp)
- Low spectral centroid: dark or muffled sounds (e.g., bass, drum)

$$\text{centroid} = \frac{\sum_{n=0}^{N-1} f(n)x(n)}{\sum_{n=0}^{N-1} x(n)},$$

where $x(n)$ represents the weighted frequency value, or magnitude, of bin number n , and $f(n)$ represents the center frequency of that bin

Python (librosa)

- `centroid = librosa.feature.spectral_centroid(y=y, sr=sr)[0]`

Frequency Domain: Spectral Centroid

Purpose:

The spectral centroid indicates where the “center of mass” of the frequency spectrum is, i.e., the frequency band where the most audio signal energy is concentrated (sound’s brightness).

Why it’s useful:

- High spectral centroid: bright or sharp sounds (e.g., cymbal, bird chirp)
- Low spectral centroid: dark or muffled sounds (e.g., bass, drum)

$$\text{centroid} = \frac{\sum_{n=0}^{N-1} f(n)x(n)}{\sum_{n=0}^{N-1} x(n)},$$

where $x(n)$ represents the weighted frequency value, or magnitude, of bin number n , and $f(n)$ represents the center frequency of that bin

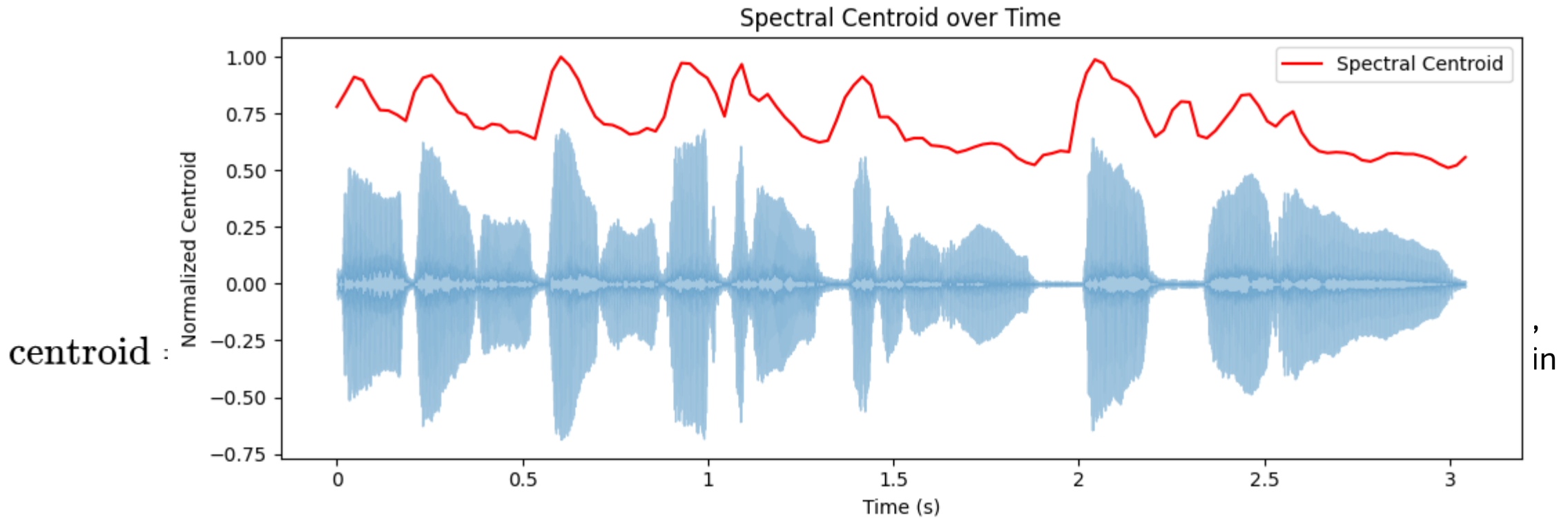
Python (librosa)

- `centroid = librosa.feature.spectral_centroid(y=y, sr=sr)[0]`

Use Cases:

- Timbre and instrument classification
- Music genre identification
- Audio segmentation

Frequency Domain: Spectral Centroid



- The trumpet's attacks contain more high-frequency energy (sharp, bright)
- The sustains are more low-frequency dominant

Frequency Domain: Spectral Flux

Purpose:

Spectral Flux measures how quickly the power spectrum changes between adjacent frames. It detects onsets and dynamic changes in audio.

Why it's useful:

- High flux = sudden change in spectrum (e.g., onset, beat)
- Low flux = steady or sustained sound

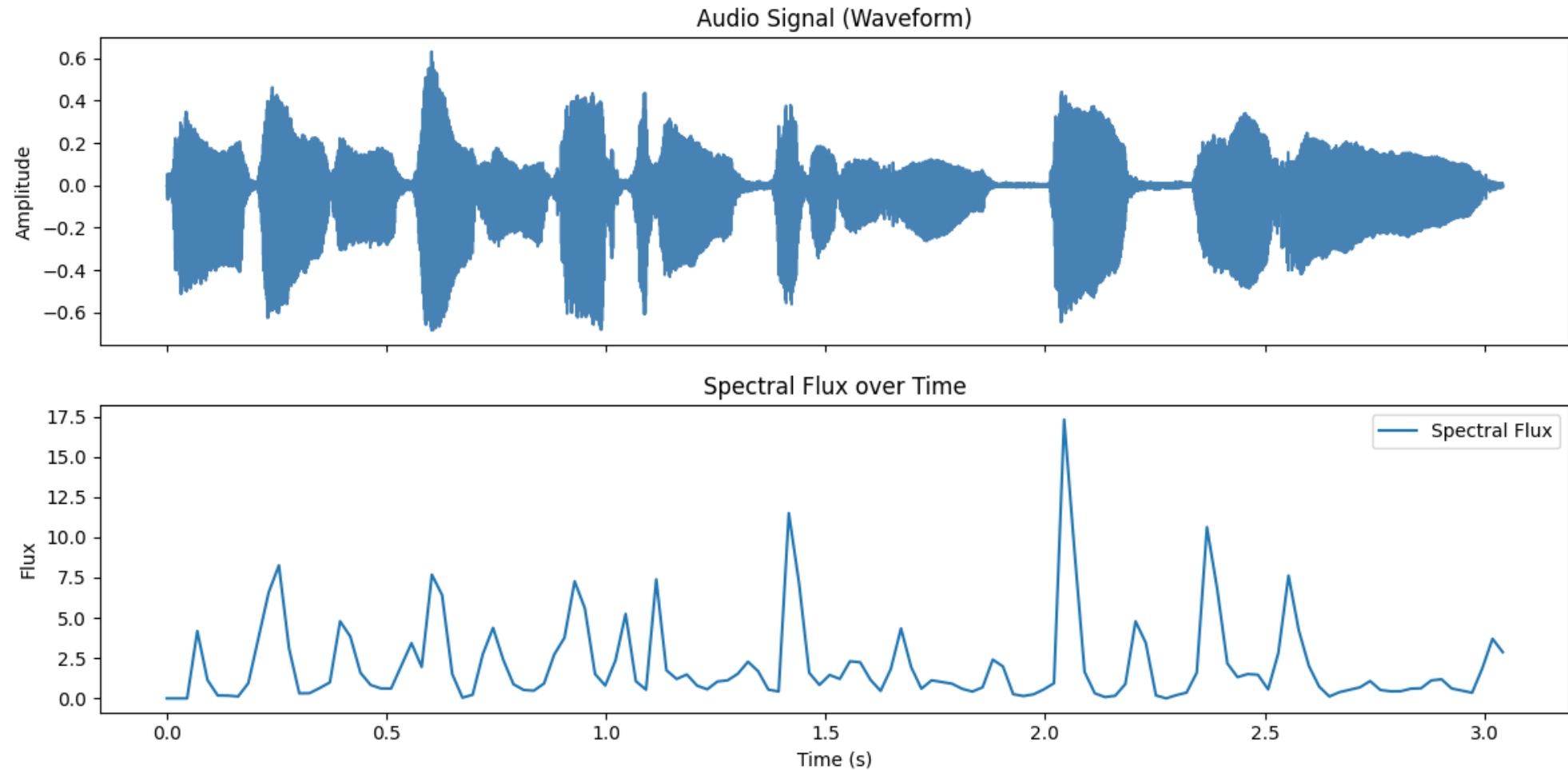
Python (librosa)

```
flux = librosa.onset.onset_strength(y=y, sr=sr)
```

Use Cases:

- Onset detection
- Beat tracking
- Rhythmic analysis

Frequency Domain: Spectral Flux



Frequency Domain: Spectral Spread

Purpose: Spectral Spread measures how spread out the frequency components are around the spectral centroid. It's a descriptor of the bandwidth of a sound.

Why it's useful:

- High spread: noisy, broadband signal (e.g., cymbal)
- Low spread: narrowband or tonal signal (e.g., sine wave)

$$\sqrt{\frac{\sum_{k=b_1}^{b_2} (f_k - \mu_1)^2 s_k}{\sum_{k=b_1}^{b_2} s_k}}$$

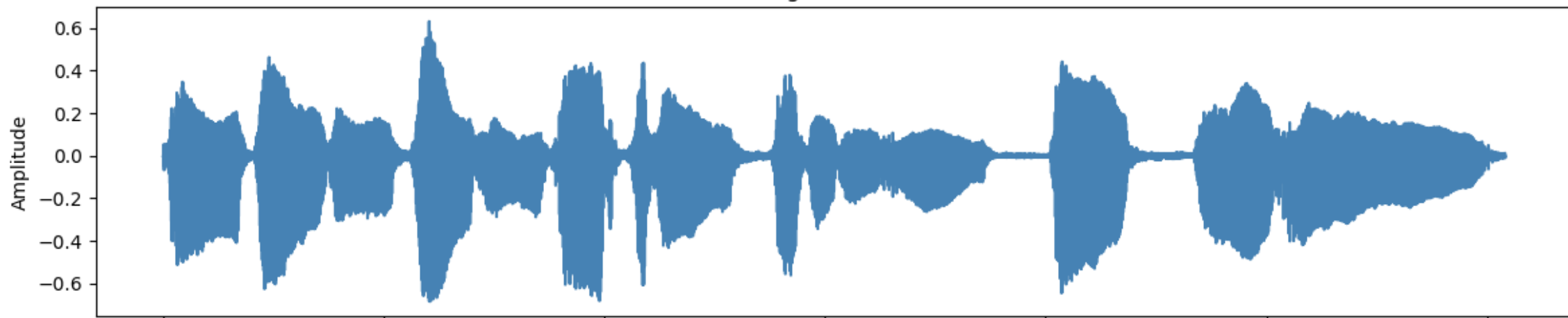
- f_k is the frequency in Hz corresponding to bin k
- s_k is the spectral value at bin k
- b_1 and b_2 are band edges, over which to calculate the spread
- μ_1 is the spectral centroid

Python (librosa)

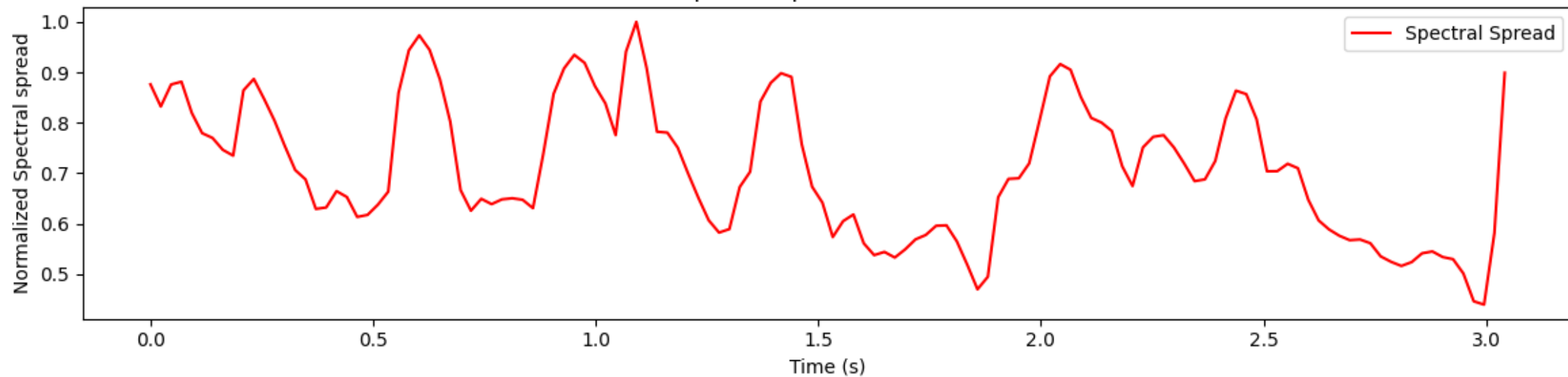
- `spread = librosa.feature.spectral_bandwidth(y=y, sr=sr)[0]`

Frequency Domain: Spectral Spread

Audio Signal (Waveform)



Spectral spread over Time



Time-Frequency Domain: Spectrogram

Purpose:

A spectrogram is a visual representation of the spectrum of frequencies in a sound as they vary with time. For ex, each color represents a musical note. Brighter colors mean the note is played more loudly.

Why it's useful:

- Shows how frequency content evolves over time
- Useful for visual analysis of speech, music, animal sounds, etc.

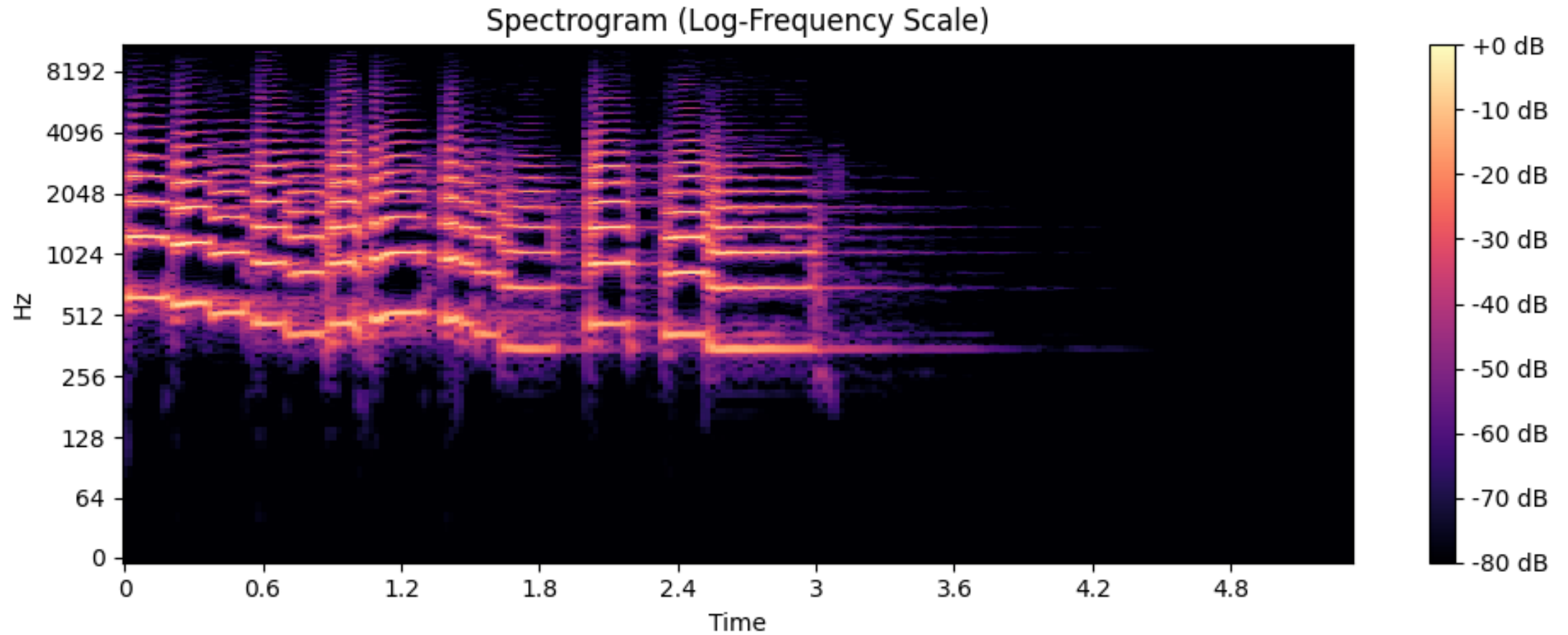
How it's created:

- Perform STFT (Short-Time Fourier Transform) on overlapping windows
- Display magnitude of frequency components over time

Python (librosa):

- Compute the Short-Time Fourier Transform (STFT) with a high number of FFT points for better frequency resolution
`D = librosa.stft(y, n_fft=4096)`
- Convert the complex-valued STFT to magnitude
`S = np.abs(D)`
- Convert to dB
`S_db = librosa.amplitude_to_db(S, ref=np.max)`
- Display Spect
`librosa.display.specshow(S_db, sr=sr, x_axis='time', y_axis='log')`

Time-Frequency Domain: Log-spectrogram



0 dB = strongest energy in the signal
-80 dB= silent

Time-Frequency Domain: Mel Spectrogram

Purpose: A Mel-Spectrogram maps the frequency axis of a spectrogram to the Mel scale — which better aligns with human auditory perception (Humans perceive frequency better at low frequencies than at higher ones).

Why it's useful:

- Focuses on perceptually relevant frequency bands
- Used in speech, music, emotion and language recognition systems

Mel Scale: Maps frequency to a scale where perceived pitch differences are equal:

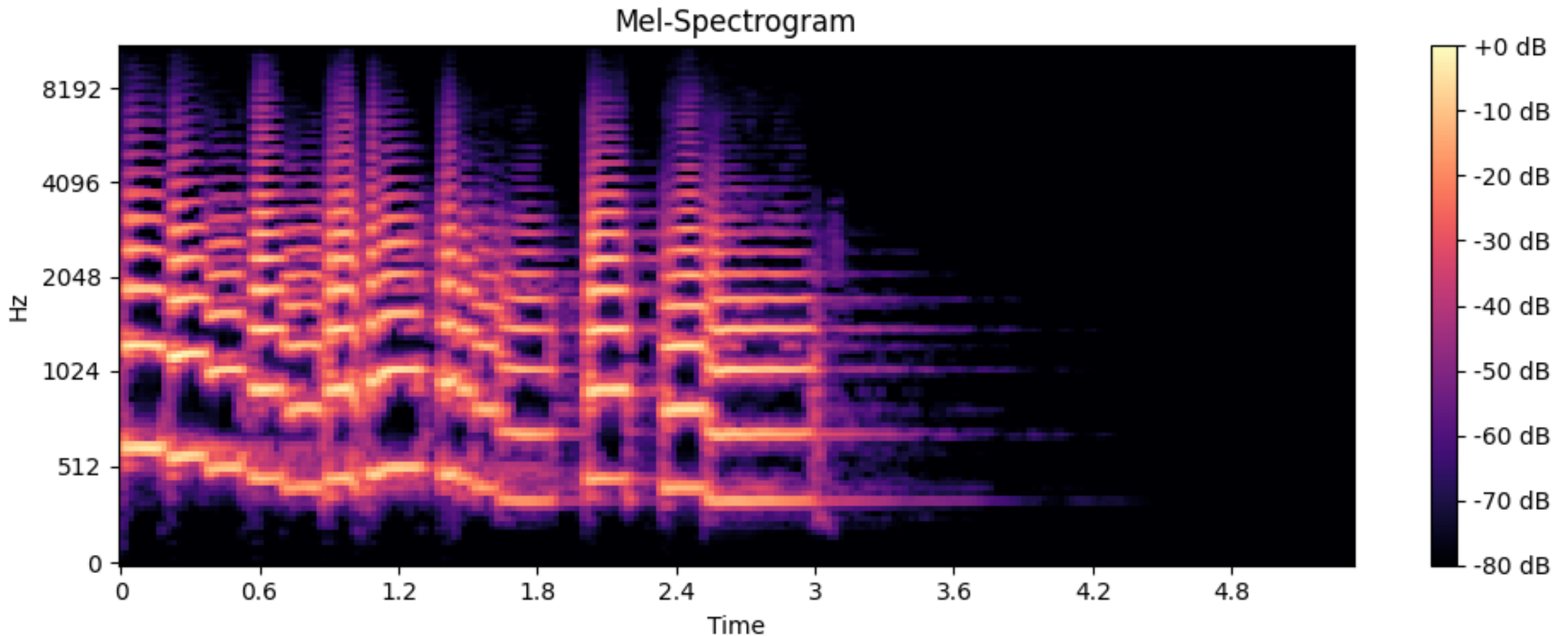
$$m = 2595 \cdot \log_{10}\left(1 + \frac{f}{700}\right)$$

Python (librosa):

```
S = librosa.feature.melspectrogram(y=y, sr=sr, n_fft=2048, hop_length=512, n_mels=128)
```

n_mels: determines the vertical resolution of the Mel spectrogram.

Time-Frequency Domain: Mel Spectrogram



Time-Frequency Domain: MFCC

Purpose:

MFCCs capture the broad shape of the spectral envelope — that is, the general distribution of energy across frequency bands — and compress this shape into a small set of coefficients. This effectively describes the timbre or tonal texture of the sound in a way that aligns with how humans perceive audio

Why it's useful:

- Most widely used feature in speech and audio recognition
- Captures perceptually meaningful spectral shape

How it works:

- Compute Mel-Spectrogram
- Take the log of the Mel energies
- Apply Discrete Cosine Transform (DCT) to decorrelate features (a sum of cosine functions oscillating at different frequencies.)

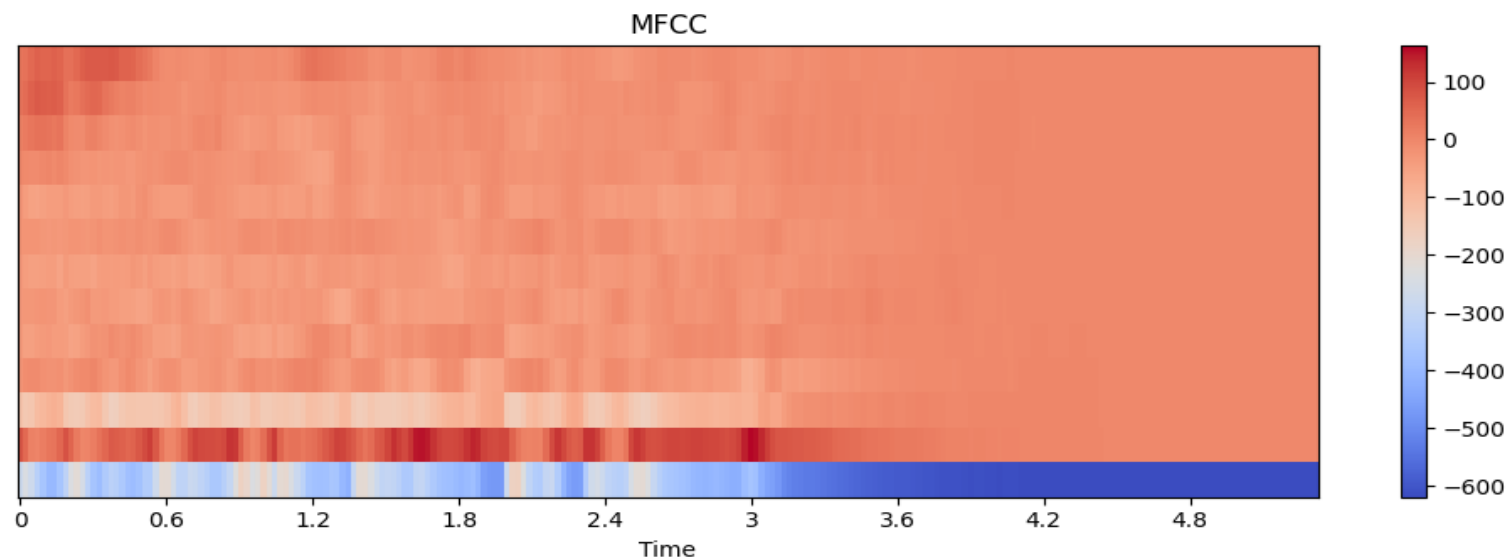
Python (librosa):

```
mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)
```

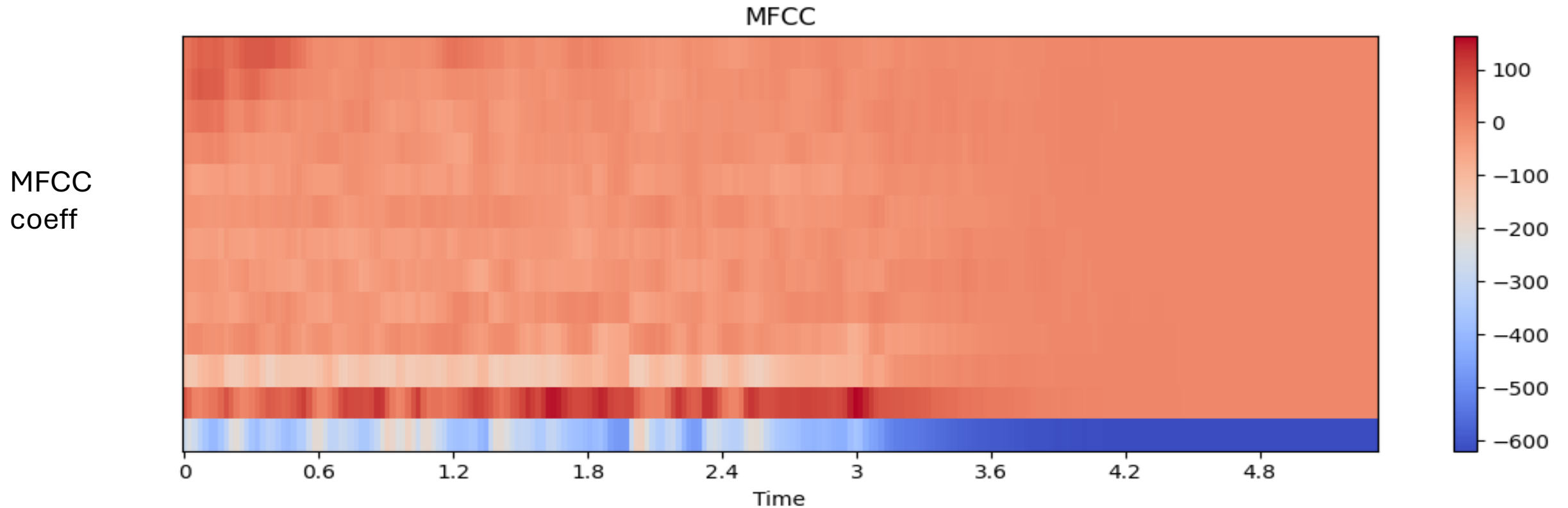
Time-Frequency Domain: MFCC

MFCC signification:

- MFCC 0 (bottom row):→ Represents overall signal energy.
- MFCC 1–2:→ Capture the overall slope of the spectral envelope (e.g., how quickly the spectrum falls off)
- MFCC 3–5:→ Represent formant-like shapes, useful in speech (vowels, resonances).
- MFCC 6+:→ Capture finer spectral detail, like rapid changes or texture — more sensitive to noise and timbre.



Time-Frequency Domain: MFCC



- Before 3s: Higher coefficients have more variation → rich, dynamic signal with changing spectral content.
- After 3s: Most coefficients become uniform and flat → Less structure or detail in the spectrum, noise, silence, or steady low-energy sound.
- Bottom row (MFCC 0) turns dark blue → confirms energy has dropped significantly

Time-Frequency Domain: CQT

Purpose:

CQT transforms a signal into a time–frequency representation like the STFT, but uses logarithmically spaced frequency bins — matching how humans perceive musical pitch.

Why it's useful:

- Better suited for musical signals than STFT
- Preserves pitch-related patterns such as chords, harmonics, or transpositions

Main idea:

- Uses geometrically spaced frequency bins (like piano notes)
- Each bin has a constant ratio between center frequency and bandwidth (Q factor)

Python Code:

```
C = np.abs(librosa.cqt(y, sr=sr))
```

Use Cases:

- Music transcription
- Pitch detection in polyphonic recordings
- Chord recognition

Time-Frequency Domain: CQT

